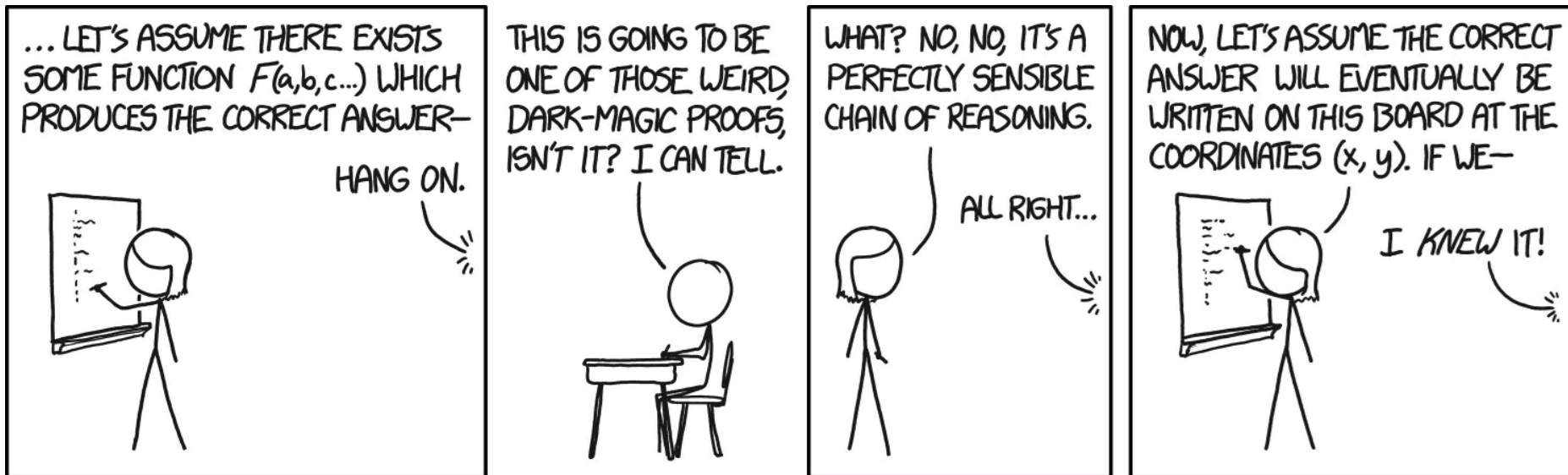


Diagonalization, Undecidability, and the Barber Paradox



Happy Do Something Nice Day

- Other Holidays:
 - World Architecture Day
 - Blue Shirt Day (anti-bullying)
 - National Consignment Day
 - World Habitat Day
 - Global James Bond Day
 - National Storytelling Day
 - World Teachers' Day
 - National Get Funky Day
 - Day of Unity
 - Child Health Day
 - Chic Spy Day
 - Rocky Mountain Oyster Day
 - National Apple Betty Day
 - National Kiss a Wrestler Day
- This week is National Walk Your Dog Week
- October is National Disability Employment Awareness Month

Exam 1 Planning

	Sunday	Monday	Tuesday	Wednes.	Thursday	Friday	Saturday
This week				Today Lec 10 Practice exams released HW 4 due			
2/15-2/21		Lec 11		Lec 12 HW 5 due			Review session #1: TBD
2/22-2/28	Review session #2: TBD	Lec 13: Exam Review	HW 6 due (TUESDAY)	Lec 14: Special Topics (not testable) 2/25: Exam 1 7-9pm			
3/1-3/7	Spring Break						

- Exam 1 covers: Algorithms (potential method, divide and conquer, DP, greedy), Computability (DFAs, TMs, decidability, Turing reductions).
 - HW 1-6, Discussion 1-6, Lec 1-12
- See “Exam 1 Meta-Post” on Piazza – coming soon – for more information
- You may bring an 8.5x11 inch reference sheet (front and back) to the exam
 - You can put anything on it you want, as long as it is created by you

HW 7 due Wed after break

Review: Decidability

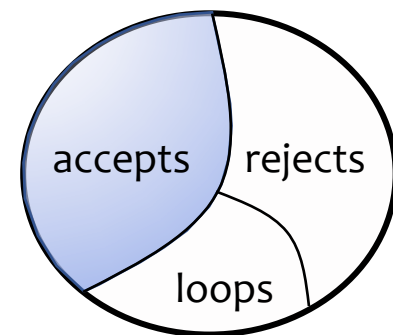
Question: What are the possible outcomes of a TM **M** on an input **x**?

Answer: **M** either (i) accepts, (ii) rejects, or (iii) it “**loops**” (**forever**)

Definition: A Turing Machine **M** **decides** a language **L** if it:

1. accepts every string in **L**, and
2. rejects every string not in **L**

(and does not loop forever on any input)



A language **L** is **decidable** if there is a TM that decides **L**.

*“there is an algorithm that solves the problem associated with **L**”*

Otherwise **L** is **undecidable**.

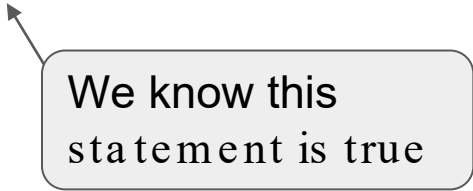
*“there is no algorithm that solves the problem associated with **L**!”*

Review: Church–Turing Thesis

“Any natural notion of being ‘algorithmically computable’ is captured by Turing Machines.”

This is not a theorem,
but everyone seems to believe it.
We will assume it’s true and proceed from there.

Anything that can be computed by Python, C++, a quantum computer, LaTeX, pseudocode, etc. can be computed by a Turing Machine.



We know this
statement is true

Question: To prove that a language L is decidable, must we write down all the details of an actual TM?

No! A TM could simulate pseudocode, so just describing what it does is sufficient!

Question: If a language L is decidable, then must $L \cup \{\epsilon\}$ also be decidable?

Yes!

Because L is decidable, there is a TM that decides it (M)

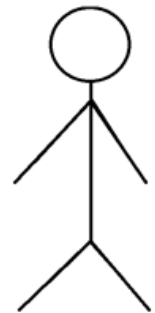
Construct TM $M'(s)$:

if $s = \epsilon$, accept

else, return $M(s)$

Note: M decides L , so it always halts

My friends and I would like to bike to lunch



How do you know there will be a problem?

There are more people than bikes, but each bike can only transport 1 person

Existence of Undecidable Languages

To show that there **exists** an **undecidable** language (i.e. a language that no TM decides), we would like to show that:

total # TMs < total # languages.

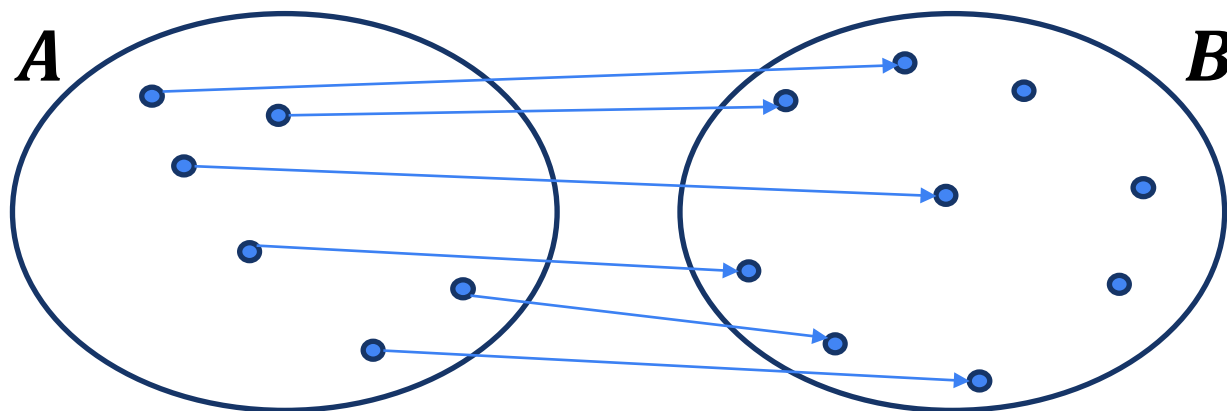
But both of these quantities are **infinite** so what does that even mean?

To infinity and beyond!



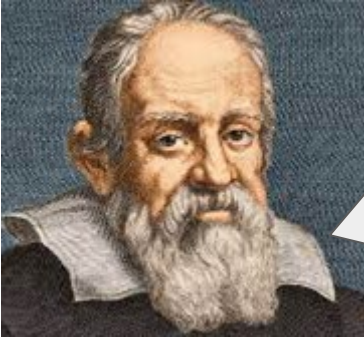
203 Review: *Countability*

- A (total) function $f: A \rightarrow B$ maps each element $a \in A$ to an element $f(a) \in B$.
- A function is *injective* (aka 1-to-1) if all elements of A map to distinct elements of B ,
i.e., $\forall a_1 \neq a_2. f(a_1) \neq f(a_2)$
- If an injective $f: A \rightarrow B$ exists, then $|A| \leq |B|$.



- A and B have the *same* cardinality, i.e., $|A| = |B|$, if we have both $|A| \leq |B|$ and $|B| \leq |A|$.

203 Review: Countability



Galileo (1638)

The attributes
“equal,” “greater,”
and “less,” are not
applicable to infinite,
but only to finite,
quantities.



Actually
yes they
are

Georg Cantor (1895)

- A set is **countable** if it is not larger than the naturals $\mathbb{N} = \{0, 1, 2, \dots\}$
- In other words, it is possible to **list every element** of the set.

Is ____ a countable set?

The integers from 1 to 10?

Yes! Finite sets are smaller than $\mathbb{N}$, so they are countable

The non-negative even numbers?

Yes! $\{0, 2, 4, 6, 8, \dots\}$

The integers? (positive, 0, and negative)

Yes! $\{0, 1, -1, 2, -2, 3, -3, 4, -4, \dots\}$

The rationals?

YES!

The reals in $(0,1)$?

a/b	1	2	3	4	5	6
1	1/1	1/2	1/3	1/4	1/5	1/6
2	2/1	2/2	2/3	2/4	2/5	2/6
3	3/1	3/2	3/3	3/4	3/5	3/6
4	4/1	4/2	4/3	4/4	4/5	4/6
5	5/1	5/2	5/3	5/4	5/5	5/6
6	6/1	6/2	6/3	6/4	6/5	6/6

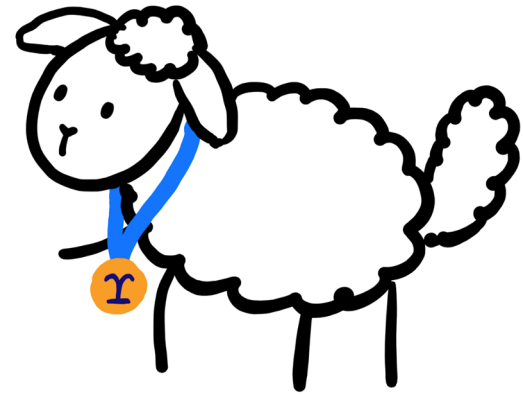
Natural Number

1
2
3
...
11
12
...

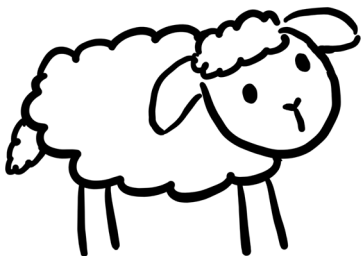
Real Number in $(0,1)$

.1
.2
.3
...
.11
.12
...

Check out my
proof that the
reals in $(0,1)$ are
countable!



My favorite real is
 $1/3$. Where in the
list is it?



Diagonalization

Theorem: the set of reals in $(0,1)$ is *uncountable*.

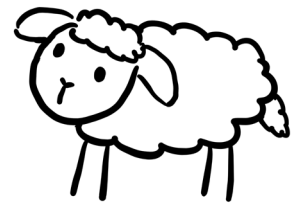
Suppose for contradiction we could write a list of all the reals in $(0,1)$.

Any $r \in (0,1)$ can be represented as an infinite decimal expansion, e.g.,
 $r = 0.95567 \dots$

$r_i = i^{\text{th}}$ real number in your list.

r_1	0.	5	6	1	2	8	...
r_2	0.	6	4	4	4	4	...
r_3	0.	1	2	3	4	5	...
r_4	0.	8	7	2	1	6	...
...							...

Our goal is to find a real in $(0,1)$ that's *not* in this supposed list!



Diagonalization

Theorem: the set of reals in $(0,1)$ is *uncountable*.

Suppose for contradiction we could write a list of all the reals in $(0,1)$.

Any $r \in (0,1)$ can be represented as an infinite decimal expansion, e.g.,
 $r = 0.95567 \dots$

$r_i = i^{\text{th}}$ real number in your list.

r_1	0.	5	6	1	2	8	...
r_2	0.	6	4	4	4	4	...
r_3	0.	1	2	3	4	5	...
r_4	0.	8	7	2	1	6	...
...							...
$r^* = r_j$	0.	6	5	5	5	...	...

Define the real number r^* to be:

$$r^*(i) = \begin{cases} 6 & \text{if } r_i(i) = 5 \\ 5 & \text{if } r_i(i) \neq 5 \end{cases}$$

Then r^* must appear in the list somewhere! Suppose $r^* = r_j$

Our goal is to find a real in $(0,1)$ that's not in this supposed list!



Diagonalization



Theorem: the set of reals in $(0,1)$ is *uncountable*.

Suppose for contradiction we could write a list of all the reals in $(0,1)$.

Any $r \in (0,1)$ can be represented as an infinite decimal expansion, e.g.,
 $r = 0.95567 \dots$

$r_i = i^{\text{th}}$ real number in your list.

		j							
		1	2	3	4	5	...	6	...
r_1	0.	5	6	1	2	8	...	6	...
r_2	0.	6	4	4	4	4	...	4	...
r_3	0.	1	2	3	4	5	...	6	...
r_4	0.	8	7	2	1	6	...	3	...
...							...		...
$r^* = r_j$	0.	6	5	5	5	...	...	???	...

Define the real number r^* to be:

$$r^*(i) = \begin{cases} 6 & \text{if } r_i(i) = 5 \\ 5 & \text{if } r_i(i) \neq 5 \end{cases}$$

Then r^* must appear in the list somewhere! Suppose $r^* = r_j$

What is $r_j(j)$?

Our goal is to find a real in $(0,1)$ that's not in this supposed list!



Strings -> Languages, (un)Countable?

How many strings are there on the alphabet $\{0,1\}$? $|\{0,1\}^*| =$

Countably infinite

$$\{0,1\}^* = \{\epsilon, \quad 0, 1, \quad 00, 01, 10, 11, \quad 000, 001, 010, 011, \dots\}$$

How many languages are there on the alphabet $\{0,1\}$?

$$|P(\{0,1\}^*)| = ??$$

Each language is a subset of $\{0,1\}^*$;

each is a (possibly infinite) set of strings

There are uncountably many Languages!

- Suppose (for contradiction) there were countably many languages on $\{0,1\}$. We could list them:

	$s_1 = \epsilon$	$s_2 = 0$	$s_3 = 1$	$s_4 = 00$	$s_5 = 01$	$s_6 = 10$	...
L_1	✓	✗	✗	✓	✓	✓	...
L_2	✗	✗	✓	✗	✓	✗	...
L_3	✓	✓	✓	✓	✗	✓	...
L_4	✗	✗	✗	✗	✗	✗	...
L_5	✓	✗	✓	✗	✗	✗	...
...	...	...	...	...	...	...	...

There are uncountably many Languages!

- Suppose (for contradiction) there were countably many languages on $\{0,1\}$. We could list them:
- Construct a language L' , where $s_i \in L' \leftrightarrow s_i \notin L_i$
- Because (by assumption) every language on $\{0,1\}$ is in the list, L' must be some L_j

	$s_1 = \epsilon$	$s_2 = 0$	$s_3 = 1$	$s_4 = 00$	$s_5 = 01$	$s_6 = 10$	...
L_1	✓	✗	✗	✓	✓	✓	...
L_2	✗	✗	✓	✗	✓	✗	...
L_3	✓	✓	✓	✓	✗	✓	...
L_4	✗	✗	✗	✗	✗	✗	...
L_5	✓	✗	✓	✗	✗	✗	...
...	...	...	...	...	...	...	...
$L' = L_j$	✗	✓	✗	✓	✓	...	...

There are uncountably many Languages!

- Suppose (for contradiction) there were countably many languages on $\{0,1\}$. We could list them:
- Construct a language L' , where $s_i \in L' \leftrightarrow s_i \notin L_i$
- Because (by assumption) every language on $\{0,1\}$ is in the list, L' must be some L_j
- Is $s_j \in L_j$? Contradiction!

	$s_1 = \epsilon$	$s_2 = 0$	$s_3 = 1$	$s_4 = 00$	$s_5 = 01$	$s_6 = 10$	...	s_j	...
L_1	☒	☒	☒	☒	☒	☒	...	☒	...
L_2	☒	☒	☒	☒	☒	☒	...	☒	...
L_3	☒	☒	☒	☒	☒	☒	...	☒	...
L_4	☒	☒	☒	☒	☒	☒	...	☒	...
L_5	☒	☒	☒	☒	☒	☒	...	☒	...
...	...	...	...	...	...	...	...	...	...
$L' = L_j$	☒	☒	☒	☒	☒	...	...	?????	

Turing Machines, (un)Countable?

Formal definition of a Turing Machine

A **Turing Machine** is a 7-tuple:

$$M = (\Sigma, \Gamma, Q, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}}, \delta)$$

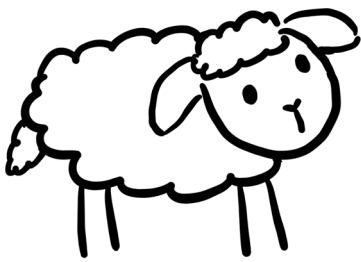
All of these
sets are
finite



- Σ is the (finite) input alphabet,
- Γ is the (finite) tape alphabet,
 $\Sigma \subseteq \Gamma$, Contains blank symbol $\perp \in \Gamma$, $\perp \notin \Sigma$
- Q is the *finite* set of states,
 - $q_{\text{start}} \in Q$ is the *initial* state,
 - $q_{\text{acc}}, q_{\text{rej}} \in Q$ are the (distinct) *accept* and *reject* states,
 $F = \{q_{\text{acc}}, q_{\text{rej}}\}$
- δ is the *transition function*.

$$\delta: \underbrace{(Q \setminus F)}_{\text{Non-terminal state}} \times \underbrace{\Gamma}_{\text{Cell contents}} \rightarrow \underbrace{Q}_{\text{Next state}} \times \underbrace{\Gamma}_{\text{Symbol to write}} \times \underbrace{\{L, R\}}_{\text{Where to move the head}}$$

A Turing Machine can
be represented with
finitely many bits!

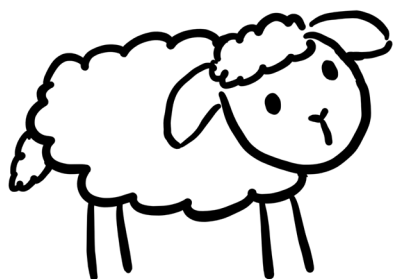


A Bag of Languages

Facts:

- There are countably many Turing Machines
So there are countably many decidable languages
(since each TM decides ≤ 1 language)
- There are uncountably many languages
- There are more languages than decidable languages, so there must be an undecidable language (in fact the VAST majority must be undecidable)

I'm going to reach into
this bag of languages and
pick one out!



What have we shown?

We showed that there *exists* an *undecidable* language.

Conclusion: there is a “problem” that *no computer can solve!*

Is it a problem that we *really wanted to solve*? Does this problem even have a comprehensible description?

Next: Identify some *explicit* undecidable language that we definitely would like to solve!

It's based on Russell's Barber Paradox.

The Barber Paradox

(aka Russell's Paradox)



“Barbara is the best barber in town! She cuts the hair of those people—and only those people—who do not cut their own hair.”

Question: *does Barbara cut her own hair?*

Who's cutting whose hair around here?

- There's a lot of cutting hair going on in this town!
- How can we keep track of who's cutting whose hair?
 - *Maybe with, like, a “hairstresser-hairstressee” matrix?*

		Hairstressees				
Hairstressers		A	B	C	D	E
	A	☒	✗	✗	✗	✗
	B	✗	✗	☒	✗	☒
	C	✗	☒	✗	✗	✗
	D	☒	✗	✗	☒	✗
	E	✗	✗	☒	✗	✗

Who's cutting whose hair around here?

- There's a lot of cutting hair going on in this town!
- How can we keep track of who's cutting whose hair?
 - *Maybe with, like, a "hairstresser-hairstressee" matrix?*

		Hairstressees				
Hairstressers		A	B	C	D	E
	A	☒	✗	✗	✗	✗
	B	✗	☒	☒	✗	☒
	C	✗	☒	☒	✗	✗
	D	☒	✗	✗	☒	✗
	E	✗	✗	☒	✗	☒
Barbara		✗	☒	☒	✗	☒

Who's cutting whose hair around here?

- There's a lot of cutting hair going on in this town!
- How can we keep track of who's cutting whose hair?
 - *Maybe with, like, a "hairstresser-hairstressee" matrix?*

		Hairstressees						
Hairstressers		A	B	C	D	E		Barbara
	A	☒	✗	✗	✗	✗		
	B	✗	☒	☒	✗	☒		
	C	✗	☒	☒	✗	✗		
	D	☒	✗	✗	☒	✗		
	E	✗	✗	☒	✗	☒		
Barbara		✗	☒	☒	✗	☒		??????

Translate the Barber Paradox into a story about TMs

- Your translation guide:
 - “**people, Barbara**” → **Turing Machines**
 - “**cut hair**” → **accept**
 - “**hairedressee**” → **descriptions of a Turing Machine / “code”**

“**Barbara** **cuts the hair** of those people—and only those people—
who do not **cut the hair** of themselves”

“**M_{BARBER}** **accepts code** of those TMs—and only those TMs—
who do not **accept code** of themselves”

- Why, precisely, is the existence of the TM **M_{BARBER}** a problem?
Is there still a paradox?